

Step 4 — Test Gate + Rollback — Solution

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

Approach 1 — the direct way

`eval_suite/tasks.py`

```
EVAL_TASKS = [  
    {"topic": "electric bikes", "check": lambda report: len(report) > 100},  
    {"topic": "remote work", "check": lambda report: "productivity" in report.lower()},  
    {"topic": "specialty coffee", "check": lambda report: len(report) > 100},  
]
```

`eval_suite/run.py`

```
from graph import graph  
from state import make_initial_state  
from eval_suite.tasks import EVAL_TASKS  
  
def run_suite():  
    all_passed = True  
    for task in EVAL_TASKS:  
        result = graph.invoke(make_initial_state(task["topic"]))  
        report = result["final_report"]  
        if not task["check"](report):  
            all_passed = False  
    return all_passed
```

`deploy_gate.py`

```
from eval_suite.run import run_suite  
  
versions = []  
  
def try_deploy(version):  
    if run_suite():
```

```
    version["status"] = "passed"
    versions.append(version)
    return True
version["status"] = "failed"
versions.append(version)
return False
```

`rollback.py`

```
from deploy_gate import versions

def rollback():
    for v in reversed(versions):
        if v["status"] == "passed":
            return v
    return None
```

This works for a simple, linear history. `rollback()` returning `None` on total failure is a real gap — a caller that forgets to check for `None` would silently treat "nothing to roll back to" as success. There's no persistence either — `versions` is an in-memory list that resets every time the process restarts.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — typed versions, persisted, every result recorded (passed or failed)

eval_suite/tasks.py

```
from typing import Callable

EVAL_TASKS: list[dict] = [
    {"topic": "electric bikes", "check": lambda report: len(report) > 100},
    {"topic": "remote work", "check": lambda report: "productivity" in report.lower},
    {"topic": "specialty coffee", "check": lambda report: len(report) > 100},
]
```

eval_suite/run.py

```
from graph import graph
from state import make_initial_state
from eval_suite.tasks import EVAL_TASKS

def run_suite() -> dict:
    results = []
    for task in EVAL_TASKS:
        result = graph.invoke(make_initial_state(task["topic"]))
        report = result["final_report"]
        passed = task["check"](report)
        results.append({"topic": task["topic"], "passed": passed})
    return {"passed": all(r["passed"] for r in results), "results": results}
```

versioning.py

```
import json
import pathlib
from datetime import datetime, timezone
from typing import TypedDict

VERSIONS_FILE = pathlib.Path("versions.json")
```

```

class Version(TypedDict):
    id: str
    status: str # "passed" | "failed" | "untested"
    created_at: str
    suite_results: list

def load_versions() -> list[Version]:
    if not VERSIONS_FILE.exists():
        return []
    return json.loads(VERSIONS_FILE.read_text())

def save_version(version: Version) -> None:
    versions = load_versions()
    versions.append(version)
    VERSIONS_FILE.write_text(json.dumps(versions, indent=2))

```

deploy_gate.py

```

from datetime import datetime, timezone
from eval_suite.run import run_suite
from versioning import Version, save_version

def try_deploy(version_id: str) -> bool:
    suite_result = run_suite()
    version: Version = {
        "id": version_id,
        "status": "passed" if suite_result["passed"] else "failed",
        "created_at": datetime.now(timezone.utc).isoformat(),
        "suite_results": suite_result["results"],
    }
    save_version(version)
    return version["status"] == "passed"

```

rollback.py

```

from versioning import load_versions

class NoPassingVersionError(Exception):
    pass

```

```
def rollback() -> dict:
    versions = load_versions()
    for version in reversed(versions):
        if version["status"] == "passed":
            return version
    raise NoPassingVersionError("no version has ever passed the suite")
```

Difference from Basic: versions persist to `versions.json`, so history survives a restart. Every version is saved — passed or failed — so `rollback()` always has the full picture, not just the ones that happened to succeed. `rollback()` raises a specific exception instead of returning `None` on total failure, forcing a caller to handle "nothing to roll back to" instead of silently getting `None` and possibly treating it as success. Still worth double-checking: does the backward scan actually skip over *multiple consecutive* failed versions correctly, and does it never accidentally treat an `"untested"` version as good enough? (It does, here — `== "passed"` is strict — but this is exactly the detail Advanced calls out explicitly and tests for.)

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — explicit backward scan, proven with a test that has 2 consecutive failures

```
# rollback.py
from versioning import load_versions

class NoPassingVersionError(Exception):
    pass

def rollback() -> dict:
    versions = load_versions() # oldest first, as stored

    for version in reversed(versions):
        if version["status"] == "passed":
            return version

    raise NoPassingVersionError("no version has ever passed the suite – nothing s
```

```
# test_api.py (excerpt) – the exact scenario the README's Step 4 test asks for
def test_rollback_skips_multiple_failed_versions():
    versions = [
        {"id": "v1", "status": "passed"},
        {"id": "v2", "status": "failed"}, # a bad change
        {"id": "v3", "status": "failed"}, # an attempted fix that also failed
    ]
    write_versions_file(versions) # test helper writing to versions.json

    result = rollback()

    assert result["id"] == "v1" # NOT v2 – "whatever came before" would wrongly
```

Expected behavior: even with 2 failed versions in a row sitting between the current (bad) state and the last good one, `rollback()` correctly walks past both and returns `v1` — proving this isn't just "return the previous version," which would have wrongly returned `v2`.

Approach 2 — same scan, plus refusing to deploy an "untested" version as if it were safe

```
# versioning.py
def get_live_version(versions: list[dict]) -> dict | None:
    """Returns the version currently marked live, or None if none is set.
    Never returns an 'untested' version – untested is not the same as safe."""
    for version in versions:
        if version.get("is_live") and version["status"] == "passed":
            return version
    return None

# deploy_gate.py
def try_deploy(version_id: str) -> bool:
    suite_result = run_suite()
    version = {
        "id": version_id,
        "status": "passed" if suite_result["passed"] else "failed",
        "created_at": datetime.now(timezone.utc).isoformat(),
        "suite_results": suite_result["results"],
        "is_live": False,
    }
    save_version(version)

    if version["status"] != "passed":
        return False

    mark_all_versions_not_live()
    version["is_live"] = True
    save_version(version) # updates the same version's is_live flag
    return True
```

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate's rollback logic is correct in the code shown, but nothing *proves* it handles multiple consecutive failures, and nothing elsewhere in the system explicitly refuses to treat "not yet tested" as "safe to use." Approach 1 adds the test that specifically proves the multi-failure case the README calls out, rather than trusting the implementation by inspection alone. Approach 2 closes a related, separate gap: `get_live_version` refuses to ever return anything whose `status` isn't literally "passed" — so even a version added directly to storage without going through `try_deploy` at all (status defaults to "untested" if you design it that way) can never accidentally become "the live version" by omission.

Which one should you actually write? Both together. Approach 1's test is the only way to actually *know* the backward-scan logic is correct instead of just believing it is — write it before trusting

rollback in anything real. Approach 2's `"untested" != "passed"` distinction is a cheap, permanent safeguard: it costs one explicit status value and an `==` check, and it closes off an entire category of "how did an unverified version end up live" incidents before they can happen.